

KopiBridge AI

Private resume-to-role alignment report

Junior AI Engineer / LLM Application Developer | 28 Sept 2026, 3:40 pm

Guidance: AI-refined

OVERALL MATCH ESTIMATE

56 %

Developing Match

This score estimates resume-to-role alignment. It does not predict interviews, offers, or hiring decisions.



ROLE FIT SUMMARY

Developing Match (56). You already have relevant foundations for an early-career LLM application path: Flask API work, a React interface, Python, SQL, Git, testing, SQLite documentation, and collaboration using Git branches. The decisive gap is not general web-development ability; it is demonstrable AI/ML product evidence. Build one small, complete proof project before positioning yourself as an LLM application developer: it should include an LLM or ML workflow, a defined evaluation check, documented failure cases, and a user-facing demo. Do not claim ML, LLM, monitoring, deployment, Docker, cloud, or MLOps experience until the supporting work exists.

RECOMMENDATION

Build one proof project first

Build one proof project first. Your existing web-development foundation is relevant, but the target role requires direct, defensible evidence of ML or LLM work and evaluation. A small complete project with a user-facing interface, clear quality checks, failure notes, README, and optionally a lightweight automated check will address the highest-value gaps more effectively than adding keywords.



Claim safety at a glance

Use these labels as a guardrail against exaggeration.

Safe to add now

Directly supported by the resume text. Keep the wording factual.

Reframe carefully

The experience appears present, but the wording or context needs care.

Needs proof first

Do not add this claim until you can show working, defensible evidence.



Existing strengths

The strongest signals already found in your resume.

API and backend development

"Built a Flask API for a campus events application."

Building a Flask API is direct, credible evidence that you can create backend functionality, which is central to applications that connect a user interface, structured data, and an AI service.

JavaScript / TypeScript

"Python, JavaScript, React, SQL, Git"

JavaScript and React align with the role's product-facing web application work and give you a practical route to building an AI feature with a usable interface.

React front-end development

"Python, JavaScript, React, SQL, Git"

Your React interface for searching campus events is concrete front-end project evidence, not only a skills-list keyword.

Python programming

"Python, JavaScript, React, SQL, Git"

Python is required for the target role and is already listed. Make it more persuasive by showing it in a completed ML or LLM workflow rather than leaving it only in the skills section.



Critical gaps

The three issues most likely to weaken this application.

1 Machine learning fundamentals

High

Missing Skill | 3-7 Days

Machine learning fundamentals are explicitly required, and there is no direct resume evidence of a dataset, model choice, evaluation metric, or error analysis.

Proof step: Complete a small ML project. Document the dataset used, the model selected, one baseline, one evaluation metric, and several failure or error cases in a README. Add it to the resume only after you can explain each element.

2 Monitoring and evaluation

High

Experience Gap | 2-5 Days

The role asks for quality evaluation and reliability after release, but the resume does not show a metric, baseline, observed failure, or monitoring approach.

Proof step: For the proof project, define a repeatable quality check. Record the baseline, the metric or pass/fail criterion, example failures, and an improvement idea. A documented manual command and expected output is acceptable before automation exists.

3 MLOps / CI-CD for ML

High

Missing Skill | 4-7 Days

MLOps or CI/CD is preferred, and there is no current evidence of automation, model evaluation workflows, or related delivery practices.

Proof step: After the project has a working evaluation check, add a lightweight GitHub Actions workflow that runs the existing tests or evaluation command. Keep the workflow and its expected result documented in the repository before mentioning CI/CD or MLOps.



Action roadmap

Start with tasks that create proof, then earn stronger resume wording.

TODAY

- ✓ Create a checklist for one proof project: user problem, input, output, test or evaluation, README, screenshot, and completion definition.
- ✓ Rewrite the current campus-events project bullet without adding unsupported outcomes.
- ✓ Identify which current claims are supported by project evidence and which require proof first.

THIS WEEK

- ✓ Build the smallest usable ML or LLM workflow for the selected problem.
- ✓ Document a dataset or source, model or workflow choice, baseline, metric or quality check, and error cases.
- ✓ Keep the repository clean with setup instructions and a command that another person can run.

THIS MONTH

- ✓ Add a simple evaluation table and failure analysis to the README.
- ✓ Run tests or evaluation consistently and, if completed, add a GitHub Actions workflow.
- ✓ Create a working user-facing demo or screenshot and prepare an explanation of the technical decisions.

LONGER TERM

- ✓ Only pursue Docker, cloud deployment, or MLOps claims after you have completed and documented the related work.
- ✓ Reassess after substantive project changes, not cosmetic resume edits.
- ✓ Apply to junior roles where your API, React, Python, SQL, testing, and newly documented AI/ML project evidence are relevant.



30-day proof plan

A practical four-week sequence tied to the highest-priority gaps.

WEEK 1



Week 1: Resume review and skill review

Keep machine learning fundamentals, monitoring, MLOps, and LLM development out of claimed skills until there is completed evidence.

Move the supported strengths—Flask API work, React, JavaScript, and Git—to the top third of the resume.

Rewrite the campus-events project using only supported facts: problem, action, technology, and verified result or [add verified result].

WEEK 2



Week 2: Build proof

Choose one narrow ML or LLM user problem and define its input, output, and success check.

Implement one working workflow rather than a broad platform.

Create a repository README covering the data or source material, approach, model or LLM workflow, prompts or guardrails where applicable, and how to run the project.

WEEK 3



Week 3: Validate, document, and deploy or prepare deployment evidence

Create a small evaluation table with a baseline, result, and failure examples.

Run the existing tests or evaluation through a documented command; add GitHub Actions if completed.

Add a screenshot or short demo evidence only if the interface works. Add a live link only if the deployment is actually complete and stable.

WEEK 4



Week 4: Truthful resume revision and application preparation

Add the proof project to the resume only after implementation, documentation, and evaluation are complete.

Use exact, factual language that names the tools and the evaluation evidence without overstating scope.

Prepare interview stories about building the workflow, testing quality, one failure case, collaboration with classmates, and the current limits of your experience.

Evidence checker

Requirement-level evidence and the safest way to use each claim.

Strong

Partial

Weak

Missing

Machine learning fundamentals

SKILLS

No direct evidence found in the resume text.

Needs proof first

Missing evidence

Python programming

SKILLS

Python, JavaScript, React, SQL, Git

Reframe carefully

Partial evidence

LLM application development

SKILLS

No Docker, cloud deployment, or RAG project yet.

Reframe carefully

Weak evidence

SQL and data manipulation

SKILLS

Python, JavaScript, React, SQL, Git

Reframe carefully

Partial evidence

Cloud platforms

TOOLS

No Docker, cloud deployment, or RAG project yet.

Reframe carefully

Partial evidence

Model or app deployment

EXPERIENCE

No Docker, cloud deployment, or RAG project yet.

Reframe carefully

Partial evidence

Computer science education

EDUCATION

BSc Computer Science, expected 2026

Reframe carefully

Partial evidence

RAG / embeddings workflow

TOOLS

Reframe carefully

Partial evidence

No Docker, cloud deployment, or RAG project yet.

API and backend development

EXPERIENCE

Safe to add now

Strong evidence

Built a Flask API for a campus events application.

JavaScript / TypeScript

SKILLS

Safe to add now

Strong evidence

Python, JavaScript, React, SQL, Git

React front-end development

SKILLS

Safe to add now

Strong evidence

Python, JavaScript, React, SQL, Git

Docker / containerisation

TOOLS

Reframe carefully

Partial evidence

No Docker, cloud deployment, or RAG project yet.

Monitoring and evaluation

EXPERIENCE

Needs proof first

Missing evidence

No direct evidence found in the resume text.

MLOps / CI-CD for ML

TOOLS

Needs proof first

Missing evidence

No direct evidence found in the resume text.



Resume wording workshop

Conservative rewrites that expose missing evidence instead of inventing it.

CURRENT WORDING

No project bullet was reliably detected.

Reframe carefully

Built a Flask API for a campus-events application with input validation, SQLite event storage, two pytest unit tests, and documented setup; collaborated with two classmates using Git branches. [Add a verified result if available.]

This preserves the documented facts while grouping the backend, data, testing, documentation, and collaboration evidence into one readable project bullet.



Interview preparation

Prepare evidence for strengths and honest language for gaps.

Five likely questions

1. Walk me through the campus-events API and the decisions behind input validation and SQLite.
2. What did your pytest tests check, and what additional tests would you add?
3. How did you coordinate work with classmates using Git branches?
4. Describe a failure case in your new ML or LLM workflow and how you measured it.
5. How would you decide whether an LLM feature is good enough to release?

Evidence to prepare

A brief architecture explanation of the existing Flask, SQLite, and React project.

The README and run instructions for the new proof project.

A small evaluation table with a baseline, result, and known failure cases.

A screenshot or working demonstration of the new user-facing workflow.

A concise explanation of your individual contribution to the class project and Git collaboration.

Weaknesses to address honestly

I have not yet completed a Docker, cloud deployment, or RAG project; I am building a small documented proof project before claiming those areas.

My current ML and LLM experience is developing, so I would explain the completed workflow, its evaluation, and its limitations rather than overstate production experience.

My existing monitoring experience is limited; I can discuss the evaluation check and documented failures I have implemented.

Do not bluff about

Do not claim ML fundamentals without a completed project that identifies data, model choice, and evaluation.

Do not claim LLM, RAG, prompt engineering, or guardrails before you can show the workflow and explain its quality checks

Do not claim cloud deployment, Docker, CI/CD, MLOps, or production monitoring based only on intent to learn them.

Do not invent user counts, performance improvements, accuracy, reliability, or presentation feedback.

Score Breakdown

Category scores are calculated locally from direct matches and adjacent evidence.

Skills

WEIGHT 35%

54%

Skills evidence is partially supported. Python, SQL, JavaScript, and React are listed, but Python and SQL need clearer project context, while ML fundamentals and LLM application development need stronger or new proof.

Experience

WEIGHT 26%

58%

Your Flask campus-events API, React interface, input validation, pytest tests, setup documentation, class presentation, and Git collaboration are useful early-career evidence. The target role also expects evaluation and post-release reliability evidence, which is not currently demonstrated.

Tools & Tech

WEIGHT 20%

50%

Git, Flask, pytest, SQLite, React, and SQL support a web-application foundation. Docker, cloud deployment, RAG, and MLOps/CI-CD are not supported by completed project evidence yet, so they should remain development goals rather than resume claims.

Education

WEIGHT 12%

64%

A BSc in Computer Science expected in 2026 aligns with the degree requirement. Strengthen the overall application by pairing the degree with a completed, explainable AI/ML proof project.

FINAL RECOMMENDATION

Build one proof project first

Build one proof project first. Your existing web-development foundation is relevant, but the target role requires direct, defensible evidence of ML or LLM work and evaluation. A small complete project with a user-facing interface, clear quality checks, failure notes, README, and optionally a lightweight automated check will address the highest-value gaps more effectively than adding keywords.

- Today, define one narrowly scoped ML or LLM project and write its completion checklist: input, output, workflow, evaluation check, failure cases, README, and demo evidence.

Practise your evidence story

Use one question from your report to prepare a specific, honest answer. Spoken claims do not establish experience or change your resume assessment.

Walk me through the campus-events API and the decisions behind input validation and SQLite.

The answer establishes a credible Flask project, but it does not yet walk through the API or explain the validation and SQLite decisions. For this question, replace the project inventory with a brief request flow, the specific validation rules, why SQLite fit the project, and what the tests covered.

► Your reviewed answer

What works

"I built a Flask API for campus events."

This is supported by the resume and gives a relevant backend starting point.

"I have not deployed the project to a cloud platform yet."

This is appropriately candid and avoids overstating deployment experience.

"I would prepare the repository and test results to explain my contribution."

Pointing to artifacts is a good instinct; a repository and test output can help substantiate your contribution.

Make it more specific

"I added input validation and two unit tests using pytest."

Explain the actual validation behavior: name the request fields checked, the invalid cases rejected, the HTTP response returned, and why those checks were needed. The pytest count and validation implementation are not supported by the supplied resume, so verify them with the repository before stating them.

"The events were stored in SQLite."

State the decision rationale: for example, whether SQLite was chosen because the project was a small local application needing a simple relational database. Only use reasons you can confirm from the project. SQLite use is not explicitly documented in the supplied resume evidence, so verify it.

"I worked with two classmates using Git branches."

Connect collaboration to your own API work—for example, identify the endpoint, validation, schema, or tests you owned. The team size and branching practice are absent from the resume evidence and need verification.

Prepare next

- 👍 Use a concise structure: purpose and endpoint flow → validation rules and rationale → SQLite rationale/schema → tests and your contribution.
- 👍 Prepare a truthful revision after checking the repository: "The Flask API handled [confirmed event operations]. I validated [confirmed fields] so [confirmed failure mode] was rejected before writing to the database. We used SQLite because [confirmed reason]. I tested [confirmed cases] with pytest."

- ✔ Bring the repository, relevant route/model files, and a passing test command or output to support the details you discuss.

Do not imply cloud deployment, Docker, LLM/RAG work, ML evaluation, or CI/CD; none is evidenced here.

Do not state the exact validation rules, pytest usage/count, SQLite rationale, team size, or Git workflow unless the repository confirms them.